

목차

1장 함수와 책임	2
1.1 함수의 정의와 경계	2
1.2 하나의 책임과 함수 크기	3
1.3 이름이 드러내는 처리 내용	4
1.4 함수 분해의 기준	5
1.5 입력과 출력의 명시	6
1.6 함수 분해 전후 구조	7
1.7 부수 효과와 예측 가능성	8
2장 인터페이스와 계약	9
2.1 인터페이스의 정의	9
2.2 계약으로서의 인터페이스	10
2.3 사전 조건과 사후 조건	11
2.4 실패를 표현하는 방법	12
2.5 계약을 적어 두는 범위	13
2.6 구현 교체와 계약 유지	14
2.7 계약이 깨지는 방식	15
3장 결합도와 의존 방향	16
3.1 결합도의 정의	16
3.2 의존 방향과 변경 전파	17
3.3 결합의 종류	18
3.4 공유 상태가 만드는 숨은 결합	19
3.5 의존 방향과 순환	20
3.6 순환 의존을 끊는 방법	21
3.7 결합을 줄이는 방법	22
4장 응집도와 모듈 구성	23
4.1 응집도의 정의	23
4.2 변경 이유로 묶기	24
4.3 같은 이름과 같은 쓰임의 구분	25
4.4 낮은 응집이 만드는 비용	26
4.5 응집도와 결합도의 관계	27
4.6 변경 이유로 묶은 모듈	28
4.7 모듈 크기를 판단하는 기준	29
5장 모듈 경계와 공개 범위	30
5.1 모듈 경계의 정의	30
5.2 공개 범위와 내부 표현	31
5.3 경계를 넘는 데이터의 형태	32
5.4 경계의 안정성과 변경 빈도	33
5.5 경계와 작업 부담	34
5.6 경계를 잘못 그은 자리	35
5.7 경계를 정하는 순서	36
6장 경계를 다시 그린 사례	37
6.1 사례의 배경과 증상	37
6.2 현재 의존 관계 파악	38
6.3 나눌 자리를 정하는 과정	39
6.4 인터페이스를 먼저 정한 이유	40
6.5 분리 절차와 확인 지점	41
6.6 옮긴 뒤 확인한 것	42
6.7 남은 결합과 다음 단계	43
7장 설계 판단 기준	44
7.1 과도한 분해의 비용	44
7.2 분해를 멈추는 기준	45
7.3 인터페이스를 넓힐 때와 좁힐 때	46
7.4 결합과 응집 점검 항목	47
7.5 경계를 바꾸는 순서	48
7.6 설계 결정을 남기는 방법	49
7.7 핵심 원칙 정리	50

1.1 함수의 정의와 경계

함수는 이름을 붙인 처리 단위다. 정해진 입력을 받아 약속된 처리를 수행하고 결과를 돌려준다. 함수를 부르는 쪽을 호출부라고 부르며, 호출부는 함수의 이름과 필요한 입력, 돌아오는 결과의 의미만 알면 된다. 내부에서 어떤 순서로 계산하는지는 호출부의 관심 밖이다.

이 구분이 함수의 경계다. 경계 바깥에는 이름과 입력과 결과가 있고, 경계 안쪽에는 처리 절차와 임시 변수와 중간 계산이 있다. 같은 이름으로 같은 결과를 돌려주는 한 안쪽은 다시 써도 된다. 경계가 분명한 함수는 읽는 사람이 안쪽을 열어 보지 않고도 호출부를 이해할 수 있게 한다.

코드를 여러 함수로 나누는 일은 줄 수를 줄이려는 것이 아니라 이런 경계를 여러 개 만드는 일이다. 경계 하나가 늘면 그 안쪽을 따로 읽고 따로 고칠 수 있는 자리가 하나 늘어난다. 이 책은 함수에서 시작해 모듈까지 같은 물음을 반복한다. 무엇을 경계 밖에 두고 무엇을 안쪽에 감출 것인가.

1.2 하나의 책임과 함수 크기

함수의 책임은 그 함수가 완결해야 하는 일의 범위다. 책임이 하나라는 말은 줄이 짧다는 뜻이 아니라, 그 함수를 고쳐야 하는 이유가 한 가지라는 뜻이다. 계산 방식이 바뀔 때만 고치는 함수와 계산 방식·출력 형식·저장 위치가 바뀔 때마다 고쳐야 하는 함수는 길이가 같아도 다르다.

책임이 여럿인 함수는 세 가지 비용을 만든다. 읽을 때 어디까지가 한 가지 일인지 매번 다시 가늠해야 하고, 고칠 때 관계없는 부분까지 함께 살펴야 하며, 다시 쓰려 할 때 필요 없는 일까지 따라온다. 셋 다 함수가 커서 생기는 문제가 아니라 섞여서 생기는 문제다.

그래서 함수 크기를 줄 수로 정해 두는 규칙은 도움이 되지 않는다. 스무 줄이라도 한 가지 일만 하면 나눌 이유가 없고, 다섯 줄이라도 서로 다른 이유로 바뀌는 두 가지 일을 담았다면 나눌 이유가 있다. 판단 기준은 길이가 아니라 고칠 이유의 수다.

1.3 이름이 드러내는 처리 내용

함수 이름은 경계 바깥에 놓인 유일한 설명이다. 호출부는 이름을 읽고 그 함수가 무엇을 하는지 판단하며, 대개 안쪽을 열어 확인하지 않는다. 따라서 이름이 실제 처리 내용을 다 담지 못하면 호출부는 잘못된 전제 위에서 코드를 작성하게 된다.

가장 흔한 어긋남은 조회처럼 보이는 이름의 함수가 값을 바꾸는 경우다. 이름은 결과를 가져오는 일만 말하는데 실제로는 저장된 값을 갱신하거나 다음 호출의 결과를 바꿔 놓는다. 호출부는 여러 번 불러도 안전하다고 판단하지만 실제로는 그렇지 않다.

이름을 지을 때 확인할 것은 두 가지다. 이름이 말하는 일 말고 다른 일을 하는가, 그리고 이름이 말하는 일을 실제로 끝까지 하는가. 두 물음에 모두 아니라고 답할 수 없다면 이름을 고치거나 함수를 나눈다. 이름을 정확히 붙이는 일은 문서를 쓰는 일이 아니라 경계를 정하는 일이다.

1.4 함수 분해의 기준

긴 처리를 나눌 때는 줄 수를 반으로 자르지 않고 처리의 단계 경계에서 자른다. 단계 경계란 앞 단계의 결과가 정리되어 다음 단계의 입력으로 넘어가는 자리다. 그 자리에서 자르면 나뉜 함수의 입력과 결과가 저절로 정해진다.

자를 자리를 찾는 실용적인 방법은 처리 과정에 붙일 이름을 먼저 말해 보는 것이다. 한 덩어리에 이름을 붙였는데 접속사가 들어가야 한다면 그 접속사 자리가 경계다. 반대로 어떤 덩어리에 이름을 붙일 수 없다면 그 덩어리는 아직 하나의 단계가 아니다.

나눌 때 함께 확인할 것은 넘겨야 하는 값의 수다. 자른 자리에서 열 개가 넘는 값을 옮겨야 한다면 그 자리는 단계 경계가 아니라 처리 도중의 임의의 지점일 가능성이 크다. 경계에서 오가는 값이 적을수록 나뉜 두 함수는 서로 독립적으로 읽힌다.

1.5 입력과 출력의 명시

함수가 쓰는 값은 두 갈래로 들어온다. 호출부가 넘겨준 입력이거나, 함수가 스스로 찾아가는 바깥의 값이다. 앞쪽은 함수의 이름 옆에 드러나 있고 뒤쪽은 본문을 읽어야만 보인다. 같은 처리를 하더라도 어느 쪽을 쓰느냐에 따라 함수의 성질이 달라진다.

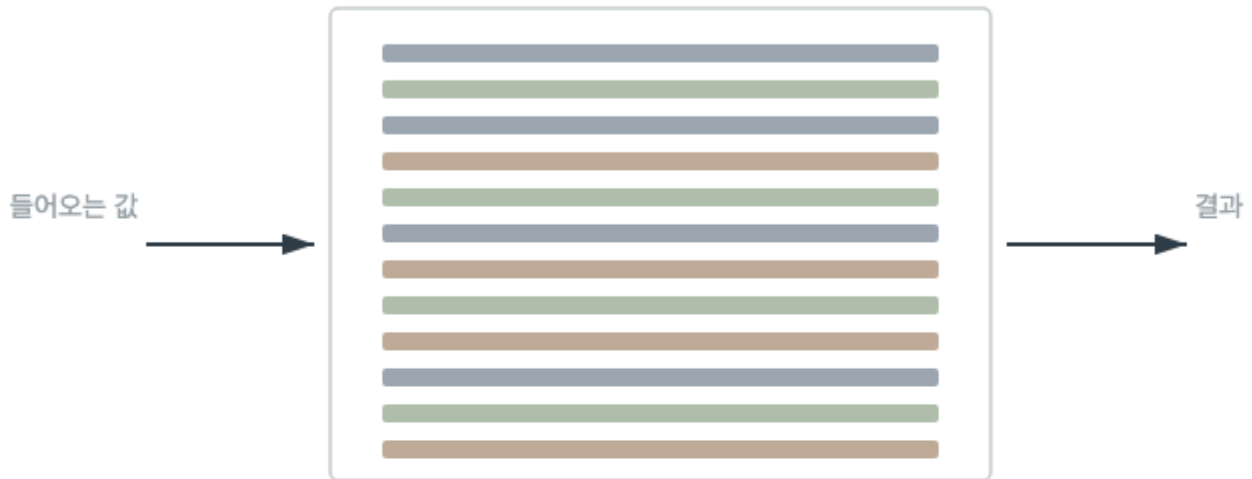
입력으로 받은 값만 쓰는 함수는 같은 입력에 같은 결과를 돌려준다. 그래서 호출부는 필요한 값을 준비하는 것만으로 결과를 통제할 수 있고, 다른 자리에서 다시 부르기도 쉽다. 반대로 바깥의 값을 직접 읽는 함수는 그 값을 누가 언제 바꾸는지까지 알아야 결과를 예측할 수 있다.

출력도 마찬가지다. 결과를 돌려주는 함수는 호출부가 그 값을 어떻게 쓸지 정하게 하지만, 결과를 어딘가에 직접 기록하는 함수는 호출부에서 결과를 볼 수 없게 만든다. 무엇이 들어가고 무엇이 나오는지 이름 옆에 드러내는 일이 함수 경계를 실제로 유지하는 방법이다.

함수 분해 전후 구조

같은 처리를 두 가지로 그린 것이다

나누기 전



나눈 뒤



나뉜 자리마다 무엇이 오가는지 드러난다

1.7 부수 효과와 예측 가능성

부수 효과는 함수가 결과를 돌려주는 일 말고 바깥에 남기는 변화다. 저장된 값을 바꾸거나, 기록을 남기거나, 넘겨받은 자료를 그 자리에서 고치는 일이 모두 여기에 든다. 부수 효과 자체가 잘못은 아니다. 어딘가에 값을 남기지 않으면 프로그램은 아무 일도 하지 못한다.

문제는 부수 효과가 이름과 결과에 드러나지 않을 때 생긴다. 호출부는 결과만 보고 판단하므로, 보이지 않는 변화가 있으면 부르는 횟수와 순서를 바꿨을 때 결과가 달라진다. 이런 함수는 따로 떼어 확인하기도 어렵다. 결과를 보는 것만으로는 그 함수가 한 일을 다 알 수 없기 때문이다.

그래서 부수 효과는 없애는 것이 아니라 모으는 것이 실용적이다. 값을 계산하는 함수와 값을 남기는 함수를 나누고, 남기는 쪽은 이름에서 그 사실을 드러낸다. 계산하는 쪽이 여럿이고 남기는 쪽이 적을수록 프로그램에서 예측하기 어려운 자리가 줄어든다.

2.1 인터페이스의 정의

인터페이스는 어떤 구성 요소를 사용하기 위해 알아야 하는 것을 모은 면이다. 함수 하나에서는 이름과 입력과 결과가 인터페이스이고, 여러 함수를 묶은 모듈에서는 바깥에서 부를 수 있는 함수의 목록과 그 각각의 약속이 인터페이스다. 크기는 달라도 역할은 같다.

인터페이스는 구현과 짝을 이루지만 같은 것이 아니다. 구현은 그 약속을 실제로 지키는 방법이고, 인터페이스는 지키기로 한 내용이다. 하나의 인터페이스에 여러 구현이 있을 수 있고, 구현을 바꿔도 인터페이스가 그대로면 사용하는 쪽의 코드는 그대로 둔다.

설계에서 인터페이스를 먼저 정하는 이유가 여기 있다. 사용하는 쪽이 무엇을 알아야 하는지 정하면 감출 것이 저절로 정해지고, 감출 것이 정해지면 안쪽을 자유롭게 고칠 수 있다. 이 책의 나머지 장은 이 면을 어디에 긋고 얼마나 넓게 잡을지를 다룬다.

2.2 계약으로서의 인터페이스

인터페이스를 계약으로 보면 양쪽의 의무가 드러난다. 사용하는 쪽은 정해진 형태의 값을 정해진 조건에 맞춰 넘길 의무가 있고, 제공하는 쪽은 그 조건이 지켜졌을 때 약속한 결과를 돌려줄 의무가 있다. 어느 한쪽이 의무를 어기면 결과는 정의되지 않는다.

계약에는 이름과 값의 형태 말고도 담아야 할 것이 있다. 값의 허용 범위, 목록이 비어 있어도 되는지, 같은 요청을 두 번 보내면 어떻게 되는지, 실패했을 때 무엇을 돌려주는지가 모두 계약의 일부다. 이런 항목은 형태만 적어서는 드러나지 않으므로 따로 밝혀야 한다.

계약을 명확히 하는 실익은 책임 소재가 갈린다는 데 있다. 조건을 어긴 값이 들어와 처리가 실패했다면 사용하는 쪽의 문제이고, 조건을 지킨 값에 약속과 다른 결과가 나왔다면 제공하는 쪽의 문제다. 경계에서 이 구분이 서면 잘못을 찾는 자리가 절반으로 줄어든다.

2.3 사전 조건과 사후 조건

사전 조건은 호출하기 전에 성립해 있어야 하는 상태다. 넘기는 값이 비어 있지 않아야 한다거나, 앞선 준비가 끝나 있어야 한다는 식이다. 사후 조건은 호출이 정상으로 끝났을 때 보장되는 상태다. 결과 목록의 순서가 정해져 있다거나, 저장이 끝나 있다는 식이다.

두 조건을 나누어 적으면 검사할 자리가 정해진다. 사전 조건은 사용하는 쪽이 맞추고 제공하는 쪽이 확인하며, 사후 조건은 제공하는 쪽이 지키고 사용하는 쪽이 기대한다. 양쪽이 같은 것을 두 번 확인하는 낭비도, 아무도 확인하지 않아 비는 자리도 이 구분에서 줄어든다.

사전 조건을 넓게 잡을수록 사용하는 쪽은 편해지고 제공하는 쪽의 처리는 복잡해진다. 반대로 좁게 잡으면 제공하는 쪽은 단순해지지만 사용하는 쪽마다 같은 준비를 반복한다. 어느 쪽이 옳다기보다 사용하는 자리가 많을수록 조건을 넓게 잡는 편이 전체 코드를 줄인다.

2.4 실패를 표현하는 방법

모든 호출이 성공하지는 않는다. 찾는 것이 없을 수도 있고, 조건을 어긴 값이 들어올 수도 있고, 안에서 부르는 다른 처리가 실패할 수도 있다. 이 셋은 성격이 달라서 같은 방식으로 알리면 사용하는 쪽이 대응을 고를 수 없다.

찾는 것이 없는 경우는 흔히 일어나는 정상 결과에 가깝다. 그래서 없음을 나타내는 값을 결과로 돌려주는 편이 다루기 쉽다. 반면 조건을 어긴 호출은 사용하는 쪽의 잘못이므로 결과에 섞지 않고 실패로 드러내야 하고, 안쪽 처리의 실패는 그대로 올려 보낼지 이 자리에서 다른 의미로 바꿔 보낼지 정해야 한다.

어느 방식을 쓰든 계약에 적어 두어야 한다는 점은 같다. 실패의 종류와 그때 돌려주는 것이 적혀 있지 않으면 사용하는 쪽은 성공만 가정한 코드를 쓰게 된다. 실패를 계약 밖에 두는 일은 실패를 없애는 일이 아니라 대비를 없애는 일이다.

2.5 계약을 적어 두는 범위

계약을 적을 때 흔한 실수는 두 방향으로 갈린다. 하나는 형태만 적고 조건을 빠뜨리는 것이고, 다른 하나는 내부 처리 절차까지 적어 두는 것이다. 앞쪽은 사용하는 쪽이 알아야 할 것을 주지 않고, 뒤쪽은 알 필요 없는 것까지 약속으로 만들어 버린다.

적어야 할 것은 사용하는 쪽의 판단을 바꾸는 항목이다. 값의 허용 범위, 실패의 종류, 순서 보장 여부, 같은 호출을 반복해도 되는지가 여기 든다. 적지 말아야 할 것은 사용하는 쪽이 알아도 행동이 달라지지 않는 항목이다. 내부에서 어떤 자료 구조를 쓰는지, 몇 단계로 나눠 처리하는지가 그렇다.

이 구분은 나중에 고칠 수 있는 범위를 정한다. 밖에 적힌 것은 사용하는 쪽이 의지해도 되는 약속이므로 바꾸려면 사용하는 쪽을 모두 확인해야 하고, 적지 않은 것은 언제든지 바꿀 수 있다. 계약을 적는 일은 설명을 늘리는 일이 아니라 고칠 자유를 남기는 일이다.

2.6 구현 교체와 계약 유지

같은 계약을 지키는 구현이 둘 이상일 수 있다. 목록을 메모리에 두고 훑는 구현과 저장소에 물어 가져오는 구현은 방식이 전혀 다르지만, 같은 입력에 같은 의미의 결과를 돌려준다면 사용하는 쪽은 둘을 구분할 필요가 없다.

교체가 실제로 가능하려면 계약에 적힌 것만으로 사용하는 쪽이 작성되어 있어야 한다. 계약에 없는 성질에 기대고 있으면 교체할 때 그 자리가 깨진다. 결과의 순서를 약속한 적이 없는데 순서를 가정하고 쓴 코드가 대표적이다. 계약 밖의 성질은 우연히 그런 것이지 약속이 아니다.

그래서 구현을 바꿀 수 있는 상태는 저절로 생기지 않는다. 계약을 좁게 적고, 사용하는 쪽이 그 범위 안에서만 쓰도록 두 번 확인해야 유지된다. 계약이 구현보다 오래 남는다는 말은 계약을 지키는 데 드는 비용을 치른 코드에서만 성립한다.

2.7 계약이 깨지는 방식

계약은 대개 한 번에 무너지지 않는다. 처음에는 잘 지켜지다가, 급한 수정이 한 번 들어가고, 그 수정이 예외적인 값을 임시로 허용하고, 그 임시가 남으면서 실제 동작과 적어 둔 약속이 갈라진다. 이때부터 계약은 코드가 아니라 기억에 남는다.

갈라졌다는 사실은 잘 드러나지 않는다. 사용하는 쪽은 대개 익숙한 몇 가지 경우만 넘기므로 어긋난 자리를 밟지 않는다. 어긋남은 새 사용처가 생기거나 값의 분포가 달라졌을 때 한꺼번에 드러나고, 그때는 원인이 된 수정이 오래전 일이라 찾기 어렵다.

막는 방법은 약속을 지키는지 자동으로 확인할 자리를 두는 것이다. 사전 조건을 어긴 호출이 조용히 지나가지 않게 하고, 실패의 종류를 결과로 드러내며, 계약을 고칠 때는 사용하는 쪽을 함께 고치는 것이다. 어긋난 것을 그때그때 드러내는 편이 나중에 한꺼번에 찾는 것보다 싸다.

3.1 결합도의 정의

결합도는 두 구성 요소가 서로에게 매여 있는 정도다. 한쪽을 고칠 때 다른 쪽도 함께 고쳐야 하는 일이 잦으면 결합이 강하고, 한쪽을 고쳐도 다른 쪽을 열어 볼 필요가 없으면 결합이 약하다. 재는 기준은 코드 사이에 선이 몇 개 그어져 있느냐가 아니라 변경이 얼마나 번지느냐다.

결합을 없앨 수는 없다. 서로 아무것도 모르는 구성 요소끼리는 함께 일할 수 없기 때문이다. 설계에서 다루는 것은 결합의 유무가 아니라 결합의 위치와 종류다. 어디에 매듭을 두고 그 매듭을 무엇으로 묶을지가 뒤이어 나올 문제다.

결합도를 낮게 유지하려는 이유는 코드가 아름다워서가 아니라 고치는 범위를 예측할 수 있어서다. 결합이 약하면 한 곳을 고칠 때 확인할 자리가 정해지고, 강하면 확인할 자리가 매번 달라진다. 예측 가능한 변경 범위가 결합도를 다루는 실질적인 목적이다.

3.2 의존 방향과 변경 전파

의존에는 방향이 있다. 값이 을의 이름과 인터페이스를 알고 부르면 값이 을에 의존한다. 이때 을이 바뀌면 값이 영향을 받지만, 값이 바뀐다고 을이 영향을 받지는 않는다. 화살표는 아는 쪽에서 알려진 쪽으로 향하고 변경의 영향은 그 반대로 흐른다.

이 성질 때문에 자주 바뀌는 것에 의존할수록 불리하다. 여러 곳에서 의존하는 구성 요소는 그만큼 바꾸기 어려워지고, 아무도 의존하지 않는 구성 요소는 자유롭게 바꿀 수 있다. 그래서 자주 바뀌는 부분이 잘 바뀌지 않는 부분에 의존하도록 방향을 잡는다.

방향을 뒤집어야 할 때는 사이에 인터페이스를 세운다. 양쪽이 서로를 직접 아는 대신 둘 다 같은 약속을 알게 하면, 화살표가 그 약속을 향하게 되어 두 구성 요소는 서로를 모르는 상태로 함께 일할 수 있다. 방향은 고정된 것이 아니라 설계로 정하는 값이다.

3.3 결합의 종류

같은 강도로 보이는 결합도 무엇을 통해 묶였는지에 따라 다루는 방법이 다르다. 가장 다루기 쉬운 것은 값을 통해 묶인 결합이다. 한쪽이 다른 쪽에 값을 넘기고 결과를 받는 형태이며, 오가는 값의 형태만 지키면 안쪽은 서로 몰라도 된다.

그보다 강한 것은 내부 표현을 통해 묶인 결합이다. 넘겨받은 자료의 속 구조를 열어 쓰거나, 상대의 저장 형식에 맞춰 값을 만들어 넣는 경우다. 이때는 상대의 안쪽이 바뀔 때마다 함께 바뀐다. 인터페이스는 그대로인데 실제로는 구현에 매여 있는 상태다.

가장 다루기 어려운 것은 처리 순서와 시점을 통해 묶인 결합이다. 어느 함수를 먼저 불러야 다음 함수가 제대로 동작하는 관계는 코드 어디에도 적혀 있지 않다. 종류를 구분하는 이유는 값을 통한 결합으로 바꿔 놓을 수 있는지 판단하기 위해서다.

3.4 공유 상태가 만드는 숨은 결합

두 함수가 서로를 부르지 않아도 같은 값을 함께 읽고 쓰면 사실상 묶여 있다. 한쪽이 그 값의 형태나 갯수 시점을 바꾸면 다른 쪽이 영향을 받기 때문이다. 이런 결합은 호출 관계를 따라가는 방식으로 보이지 않아서 구조를 그려 놓고도 놓치기 쉽다.

숨은 결합은 편의에서 시작되는 경우가 많다. 값을 여러 곳에서 필요로 하니 공용 자리에 두고, 누구나 읽게 하고, 그러다 쓰기도 하게 된다. 이 시점부터 그 값을 언제 누가 바꾸는지가 프로그램 전체의 문제가 되고, 문제가 생겼을 때 관련된 자리를 특정할 수 없다.

드러내는 방법은 두 가지다. 값을 공용 자리에서 빼서 필요한 곳에 인자로 넘기거나, 그 값을 다루는 코드를 한 모듈 안으로 모아 바깥에서는 정해진 함수로만 건드리게 하는 것이다. 어느 쪽이든 보이지 않던 결합을 인터페이스 위로 올리는 일이다.

의존 방향과 순환

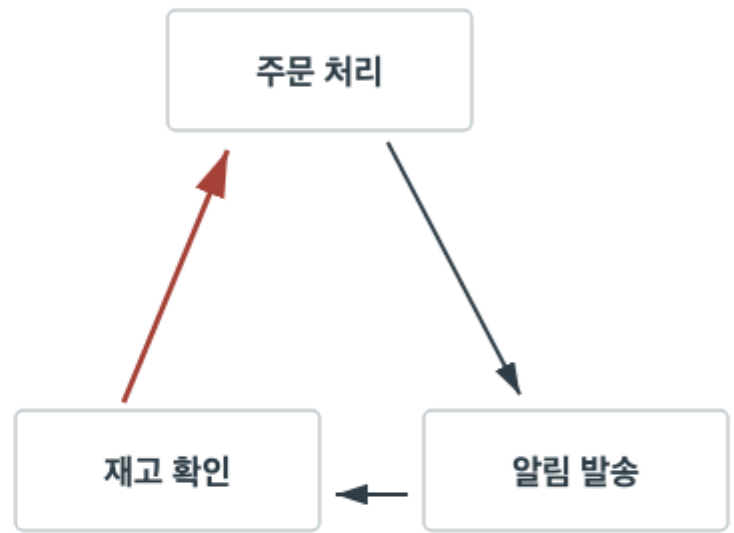
화살표는 아는 쪽에서 알려진 쪽으로 향한다

한쪽으로 흐르는 의존



아래쪽은 위쪽을 모른다

되돌아오는 의존



고칠 곳을 정하려면
셋을 함께 열어야 한다

화살표가 돌아오면 경계가 하나로 합쳐진다

3.6 순환 의존을 끊는 방법

순환 의존은 갑이 을을 알고 을이 다시 갑을 아는 상태다. 이 상태에서는 둘 중 하나만 따로 읽거나 따로 옮기거나 따로 확인할 수 없다. 경계가 둘로 보이지만 실제로는 하나이며, 셋 이상이 얹히면 그 전체가 한 덩어리가 된다.

끊는 첫 번째 방법은 되돌아오는 쪽이 정말 필요한지 다시 보는 것이다. 을이 갑을 부르는 이유가 결과를 알리기 위해서라면, 알리는 대신 결과를 돌려주고 갑이 이어서 처리하게 바꿀 수 있다. 대개의 순환은 알림을 호출로 만든 자리에서 생긴다.

두 번째 방법은 되돌아오는 화살표가 향하는 지점을 인터페이스로 바꾸는 것이다. 을은 갑을 직접 알지 않고 약속만 알고, 갑이 그 약속을 지키는 쪽으로 참여한다. 세 번째는 두 쪽이 함께 쓰는 부분을 떼어 별도 모듈로 옮기는 것이다. 어느 방법이든 목적은 화살표를 한 방향으로 만드는 데 있다.

3.7 결합을 줄이는 방법

결합을 줄이는 첫 수단은 오가는 것을 줄이는 것이다. 넘기는 값이 적을수록, 그리고 그 값이 상대의 내부 구조가 아니라 단순한 형태일수록 결합은 얇아진다. 상대의 자료를 통째로 넘기는 대신 실제로 쓰는 값만 넘기는 것만으로도 매듭 하나가 사라진다.

두 번째 수단은 아는 대상을 바꾸는 것이다. 구체적인 구현을 알던 자리를 인터페이스를 아는 자리로 바꾸면, 그 아래에서 무엇이 바뀌든 사용하는 쪽은 그대로 남는다. 이 방법은 결합을 없애지 않고 잘 바뀌지 않는 곳으로 옮기는 것에 가깝다.

세 번째 수단은 순서에 대한 의존을 값에 대한 의존으로 바꾸는 것이다. 먼저 불러야 한다는 약속을 말로 남기는 대신, 앞 단계의 결과를 다음 단계의 입력으로 만들면 순서가 코드에 드러난다. 셋 다 결합을 보이게 만들어 다루기 쉬운 종류로 바꾸는 일이다.

4.1 응집도의 정의

응집도는 한 모듈 안에 모인 것들이 서로 얼마나 붙어 있는지를 뜻한다. 안에 있는 함수들이 같은 일의 부분들이면 응집이 높고, 저마다 다른 일을 하는 것들이 한자리에 모여 있으면 응집이 낮다. 결합도가 모듈 사이의 성질이라면 응집도는 모듈 안의 성질이다.

응집을 재는 손쉬운 방법은 모듈의 이름을 말해 보는 것이다. 안에 든 것을 모두 포함하는 이름을 붙일 수 있으면 응집이 높다. 이름에 '그리고'나 '기타'가 들어가야 한다면, 또는 이름이 '도구 모음'처럼 무엇이든 담을 수 있는 말이 된다면 응집이 낮은 것이다.

응집도가 낮으면 모듈 이름으로 위치를 짐작할 수 없어 찾는 데 시간이 들고, 필요한 함수 하나 때문에 관계없는 것들까지 함께 딸려 온다. 응집은 보기 좋게 정리하는 문제가 아니라 무엇을 어디서 찾을지, 무엇을 함께 옮길지 정하는 문제다.

4.2 변경 이유로 묶기

무엇을 한 모듈에 둘지 정하는 가장 실용적인 기준은 함께 고쳐지는가다. 같은 요구가 바뀔 때 언제나 함께 손대는 코드는 한자리에 있는 편이 낫고, 서로 다른 이유로 바뀌는 코드는 나뉘어 있는 편이 낫다. 함께 바뀌는 것을 흩어 놓으면 한 번의 수정이 여러 자리를 돌아다니게 된다.

이 기준은 코드의 생김새로 묶는 방식과 자주 어긋난다. 모양이 비슷한 함수들을 한곳에 모아 두면 정리된 것처럼 보이지만, 그 함수들이 서로 다른 요구에 매여 있다면 수정은 늘 여러 모듈에 걸친다. 겉모습이 아니라 바뀌는 이유가 묶는 기준이다.

변경 이유를 알아내려면 지금까지의 수정 이력을 보는 것이 가장 빠르다. 매번 함께 고친 파일들은 이미 하나의 단위였던 것이고, 한 번도 함께 고쳐진 적 없는 것들이 한 모듈에 있다면 그 모듈은 둘로 나뉠 준비가 되어 있다.

4.3 같은 이름과 같은 쓰임의 구분

이름이 같다고 같은 것은 아니다. 서로 다른 업무에서 쓰는 두 자료가 우연히 같은 이름을 가질 수 있고, 처음에는 담긴 값도 같을 수 있다. 이때 둘을 하나로 합치면 당장은 코드가 줄지만, 두 업무의 요구가 갈라지는 순간 합쳐진 자리가 양쪽의 조건을 모두 떠안는다.

구분하는 방법은 바뀌는 이유를 각각 물어보는 것이다. 한쪽의 요구가 바뀔 때 다른 쪽도 반드시 함께 바뀌어야 한다면 같은 것이고, 한쪽만 바뀔 수 있다면 이름이 같아도 다른 것이다. 같은 값을 담고 있다는 사실은 지금의 상태일 뿐 앞으로의 약속이 아니다.

반대 방향의 실수도 있다. 이름이 다르다는 이유로 실제로는 같은 것을 두 자리에 두는 경우다. 이때는 한쪽만 고치고 다른 쪽을 빠뜨리는 일이 생긴다. 판단은 이름의 일치보다 함께 바뀌어야 하는지 여부로 내린다.

4.4 낮은 응집이 만드는 비용

응집이 낮은 모듈은 처음에는 편리해 보인다. 어디에 둘지 정하기 어려운 함수를 모두 받아 주기 때문이다. 이런 모듈은 대개 이름부터 넓다. 이름이 넓으면 새 함수를 넣을 때 아무 저항이 없고, 저항이 없으니 계속 커진다.

비용은 세 자리에서 나온다. 첫째, 이 모듈은 거의 모든 곳에서 쓰이므로 여기가 바뀌면 영향 범위를 가늠할 수 없다. 둘째, 필요한 함수 하나 때문에 이 모듈 전체를 끌어오게 되어 의존 관계가 넓어진다. 셋째, 안에 든 함수들이 서로 무관하므로 이 모듈만 따로 확인하는 일이 의미를 잃는다.

정리하는 방법은 이름을 다시 붙이는 데서 시작한다. 안에 든 함수들을 쓰임에 따라 묶고 각 묶음에 좁은 이름을 붙여 본다. 이름이 붙는 묶음은 그대로 떼어 내고, 어디에도 붙지 않는 것만 남긴다. 남은 것이 적을수록 다음 정리가 쉬워진다.

4.5 응집도와 결합도의 관계

응집도와 결합도는 따로 다루지만 실제로는 함께 움직인다. 관련 있는 것을 한 모듈에 모으면 그 안에서 오가던 호출이 모듈 내부로 들어가므로 모듈 사이의 선이 줄어든다. 응집을 높이는 일이 결합을 줄이는 일이 되는 것은 이 때문이다.

반대로 모듈을 잘게 나누기만 하면 두 지표가 어긋난다. 함께 바뀌는 것을 갈라놓으면 각 모듈은 작아지지만 그 사이를 잇는 호출과 값 전달이 늘어난다. 모듈 수가 늘었는데 고치는 범위는 그대로거나 오히려 넓어진다면 이 경우다.

그래서 두 지표는 따로 보지 않고 한 물음으로 합쳐서 본다. 이 경계를 그었을 때 한 가지 요구가 바뀌면 몇 개의 모듈을 열어야 하는가. 답이 하나에 가까울수록 응집이 높고 결합이 약한 배치이며, 여럿이라면 경계를 잘못 그은 것이다.

변경 이유로 묶은 모듈

같은 코드를 두 가지 기준으로 나눈 것이다

생김새로 묶은 배치



한 가지 요구가 바뀌면 세 네모를 연다

변경 이유로 묶은 배치



한 가지 요구가 바뀌면 한 네모를 연다

색은 함께 바뀌는 코드를 뜻한다

4.7 모듈 크기를 판단하는 기준

모듈이 커졌다는 이유만으로 나누는 것은 근거가 되지 않는다. 안에 든 것이 모두 같은 이유로 바뀐다면 그 모듈은 크더라도 하나의 단위다. 반대로 작은 모듈이라도 두 가지 이유로 바뀐다면 나눌 이유가 있다. 크기는 결과이지 기준이 아니다.

나눌 때가 되었다는 신호는 몇 가지로 드러난다. 한 모듈의 서로 다른 부분을 각기 다른 사람이 고치면서 계속 부딪히거나, 수정 이력에서 언제나 함께 바뀌는 파일 묶음이 두 갈래로 갈리거나, 모듈 이름으로 안에 든 것의 절반만 설명되는 경우다.

나누기 전에 확인할 것은 나눈 뒤 둘 사이에 오갈 값의 양이다. 나누자마자 서로를 계속 부르고 많은 값을 주고받아야 한다면 그 자리는 경계가 아니다. 나눈 뒤 오가는 것이 적어야 나눈 값이 있다.

5.1 모듈 경계의 정의

모듈 경계는 그 모듈이 무엇을 책임지고 무엇을 책임지지 않는지를 가르는 선이다. 선 안쪽에는 그 책임을 수행하는 함수와 자료가 있고, 선 위에는 바깥에서 쓸 수 있는 인터페이스가 있으며, 선 바깥에는 그 인터페이스만 아는 사용자가 있다.

경계는 폴더나 파일 구분과 같은 말이 아니다. 파일이 나뉘어 있어도 서로의 안쪽을 자유롭게 열어 쓴다면 경계는 없는 것이고, 한 자리에 모여 있어도 정해진 함수로만 오간다면 경계는 있는 것이다. 경계를 만드는 것은 배치가 아니라 접근을 제한하는 약속이다.

경계를 그으면 세 가지가 정해진다. 안쪽에서 자유롭게 바뀌어도 되는 범위, 바깥에서 의지해도 되는 약속, 그리고 문제가 생겼을 때 책임을 물을 자리다. 앞의 두 장에서 다룬 결합과 응집은 결국 이 선을 어디에 그을 것인가로 모인다.

5.2 공개 범위와 내부 표현

공개 범위는 모듈이 바깥에 내보이는 것의 목록이다. 좁게 잡으면 사용하는 쪽이 할 수 있는 일이 줄지만 안쪽을 고칠 자유가 커지고, 넓게 잡으면 반대가 된다. 처음에 좁게 시작해 필요할 때 넓히는 편이 반대 순서보다 쉽다. 한 번 공개한 것은 되돌리기 어렵기 때문이다.

특히 조심할 것은 내부 표현이 공개 범위에 섞여 나가는 경우다. 모듈이 안에서 쓰는 자료 구조를 그대로 돌려주면 사용하는 쪽은 그 구조에 맞춰 코드를 쓴다. 이 시점부터 그 구조는 내부가 아니라 약속이 되며, 바꾸려면 사용하는 쪽을 모두 고쳐야 한다.

공개할지 정할 때는 이 물음을 쓴다. 바깥에서 이것을 알아야 자기 일을 할 수 있는가, 아니면 지금 편해서 쓰는가. 뒤쪽이라면 감추고, 대신 필요한 일을 대신해 주는 함수를 하나 내보낸다. 감추는 일은 막는 일이 아니라 무엇을 약속으로 삼을지 고르는 일이다.

5.3 경계를 넘는 데이터의 형태

경계를 넘는 값은 두 모듈이 함께 아는 것이므로 어느 한쪽의 사정에 맞추면 곧 결합이 된다. 보내는 쪽의 내부 구조를 그대로 넘기면 받는 쪽이 그 구조에 매이고, 받는 쪽의 편의에 맞춰 만들면 보내는 쪽이 받는 쪽의 사정을 알아야 한다.

그래서 경계를 넘는 값은 양쪽 어느 쪽의 내부도 아닌 형태로 정한다. 그 업무에서 뜻이 분명한 항목만 담고, 내부에서만 쓰는 임시 값이나 처리 단계 표시는 넣지 않는다. 담긴 항목이 적을수록 양쪽이 함께 지켜야 할 약속이 줄어든다.

값을 넘길 때 함께 정해야 할 것이 하나 더 있다. 넘긴 값을 받는 쪽이 고쳐도 되는지다. 고쳐도 된다면 보내는 쪽은 그 값을 다시 쓰지 못하고, 고치면 안 된다면 그 약속을 계약에 적어야 한다. 적지 않으면 양쪽이 서로 다르게 가정한 채로 동작한다.

5.4 경계의 안정성과 변경 빈도

경계는 자주 바뀌는 자리에 두면 값을 하지 못한다. 경계를 하나 그으면 그 선 위의 약속은 양쪽이 함께 지켜야 하므로, 약속 자체가 자주 바뀌면 나눈 두 모듈은 늘 함께 수정된다. 나누기 전보다 손이 더 가는 상태가 된다.

그래서 경계는 업무에서 오래 유지되는 개념 위에 긋는다. 화면의 생김새나 저장 방식은 자주 바뀌지만 그 업무가 다루는 대상과 규칙은 상대적으로 오래간다. 오래가는 것을 인터페이스로 삼고 자주 바뀌는 것을 안쪽에 두면 안쪽을 여러 번 고쳐도 바깥이 흔들리지 않는다.

안정성은 저절로 생기지 않고 확인해야 한다. 지난 수정에서 이 인터페이스가 몇 번 바뀌었는지 세어 보면 된다. 자주 바뀐 인터페이스는 경계가 잘못 그어졌다는 신호이고, 한 번도 바뀌지 않은 인터페이스는 안쪽을 마음대로 고칠 수 있게 해 준 자리다.

5.5 경계와 작업 분담

경계는 코드를 읽는 순서만 정하는 것이 아니라 사람들이 부딪히는 자리도 정한다. 두 사람이 서로 다른 모듈을 맡고 그 사이의 인터페이스가 정해져 있으면, 각자 안쪽을 고치는 동안 상대의 진행을 기다릴 일이 없다. 합칠 때 충돌하는 자리도 인터페이스 근처로 좁혀진다.

반대로 경계가 흐리면 같은 파일의 같은 부분을 여러 사람이 동시에 고치게 된다. 이때 생기는 비용은 합치는 수고만이 아니다. 상대가 무엇을 바꾸고 있는지 모른 채 각자 가정을 세우므로, 합친 뒤에야 두 가정이 어긋난다는 사실이 드러난다.

그래서 여럿이 함께 일할 때는 인터페이스를 먼저 정하고 각자 안쪽으로 들어가는 순서가 유리하다. 먼저 정해 둔 약속이 있으면 상대의 코드가 아직 없어도 자기 쪽을 진행할 수 있고, 나중에 붙였을 때 어긋난 자리가 약속과 다른 곳으로 한정된다.

5.6 경계를 잘못 그은 자리

경계를 그었는데도 나아지지 않는 배치에는 공통된 모습이 있다. 하나는 한 가지 요구가 바뀔 때마다 선 양쪽을 함께 고치는 경우다. 이때는 선이 업무의 결이 아니라 코드의 생김새를 따라 그어져 있다. 선을 지우고 다시 긋는 편이 빠르다.

다른 하나는 인터페이스가 지나치게 자세한 경우다. 안쪽 처리 단계마다 함수가 하나씩 공개되어 있으면 사용하는 쪽이 순서를 알아야 하고, 결국 안쪽 사정을 다 아는 상태가 된다. 선은 있지만 감춘 것이 없다.

세 번째는 오가는 값이 너무 많은 경우다. 한 번 부를 때마다 열 개가 넘는 값을 넘기고 그중 여럿이 다른 값에 따라 뜻이 달라진다면, 두 모듈은 사실상 하나의 처리를 나눠 가진 것이다. 셋 다 경계의 자리가 아니라 경계의 근거를 다시 볼 일이다.

5.7 경계를 정하는 순서

경계를 정할 때는 코드부터 옮기지 않고 순서를 밟는다. 첫째로 이 부분이 바뀌는 이유를 적어 본다. 이유가 둘 이상이면 그 이유의 수가 나눌 후보의 수가 된다. 이유를 적지 못하면 아직 나눌 근거가 없는 것이다.

둘째로 각 이유에 속하는 함수와 자료를 모아 본다. 이때 어느 쪽에도 속하지 않거나 양쪽에 걸치는 것이 나오는데, 이런 것들이 실제 경계의 모양을 정한다. 걸치는 것이 많으면 그 자리는 아직 나눌 때가 아니다.

셋째로 나눌 뒤 두 쪽이 주고받아야 할 것을 적어 인터페이스 초안을 만든다. 초안이 몇 개의 함수와 단순한 값으로 정리되면 그 자리는 경계이고, 정리되지 않으면 첫째 단계로 돌아간다. 코드를 옮기는 일은 이 세 단계를 지난 뒤에 시작한다.

6.1 사례의 배경과 증상

한 서비스에서 주문을 다루는 모듈이 오랫동안 커졌다. 처음에는 주문을 받아 저장하는 일만 했지만 시간이 지나면서 재고를 줄이고, 알림 문구를 만들고, 정산에 넘길 자료를 준비하는 일까지 이 모듈로 들어왔다. 새 일이 생길 때마다 관련 있어 보였기 때문이다.

증상은 세 가지로 나타났다. 알림 문구만 바꾸는 수정에도 주문 저장 부분을 함께 확인해야 했고, 여러 사람이 같은 파일을 동시에 고치는 일이 잦아졌으며, 이 모듈을 따로 확인하려면 재고와 정산 쪽 준비까지 갖춰야 했다.

이 장은 이 모듈의 경계를 다시 그은 과정을 순서대로 따라간다. 앞 장들에서 다룬 변경 이유, 의존 방향, 인터페이스 선행 정의가 실제로 어떤 순서로 쓰이는지 보이는 것이 목적이며, 결과보다 각 단계에서 무엇을 보고 무엇을 정했는지에 초점을 둔다.

6.2 현재 의존 관계 파악

첫 작업은 이 모듈이 무엇을 알고 있고 누가 이 모듈을 아는지 적는 일이었다. 바깥에서 부르는 함수를 모두 찾아 호출하는 쪽과 함께 목록으로 만들고, 이 모듈이 부르는 다른 모듈도 같은 방식으로 적었다. 이 목록이 나중에 무엇이 깨질 수 있는지 판단하는 기준이 되었다.

목록을 만들면서 호출로 드러나지 않는 의존이 둘 나왔다. 하나는 여러 곳에서 함께 읽고 쓰던 설정 값이었고, 다른 하나는 주문 저장에 끝난 뒤에만 재고 처리를 불러야 한다는 순서 약속이었다. 둘 다 코드에는 적혀 있지 않았다.

이 단계에서는 아무것도 고치지 않았다. 지금 상태를 적는 일과 고치는 일을 섞으면, 나중에 문제가 생겼을 때 원래 그랬던 것인지 이번에 바뀐 것인지 가릴 수 없기 때문이다. 목록은 나눈 뒤에 다시 비교할 기준으로 남겼다.

6.3 나눌 자리를 정하는 과정

나눌 자리의 후보는 셋이었다. 재고 처리를 떼는 안, 알림 문구 생성을 떼는 안, 정산 자료 준비를 떼는 안이다. 셋 다 나눌 근거가 있었으므로 수정 이력을 기준으로 순서를 정했다. 지난 반년 동안 가장 자주 고쳐졌고 다른 부분과 함께 고쳐진 적이 가장 적은 것은 알림 문구였다.

다음으로 각 후보를 떼었을 때 오갈 값을 적어 보았다. 알림 문구는 주문 번호와 상태만 있으면 만들 수 있어 넘길 값이 적었고, 재고 처리는 주문 항목 전체와 처리 시점 정보가 필요해 값이 많았다. 정산 자료는 그 중간이었다.

자주 바뀌면서 오가는 값이 적은 알림 문구를 먼저 떼기로 했다. 자주 바뀌는 쪽을 먼저 떼면 나머지 부분의 수정 횟수가 곧바로 줄고, 오가는 값이 적으면 인터페이스를 정하는 데 드는 시간도 짧기 때문이다.

6.4 인터페이스를 먼저 정한 이유

코드를 옮기기 전에 새 모듈의 인터페이스를 먼저 적었다. 함수는 둘이었다. 주문 번호와 상태를 받아 문구를 돌려주는 함수와, 문구 종류의 목록을 돌려주는 함수다. 각 함수에 넘길 값의 조건과 돌려줄 값의 형태, 조건을 어겼을 때의 결과를 함께 적었다.

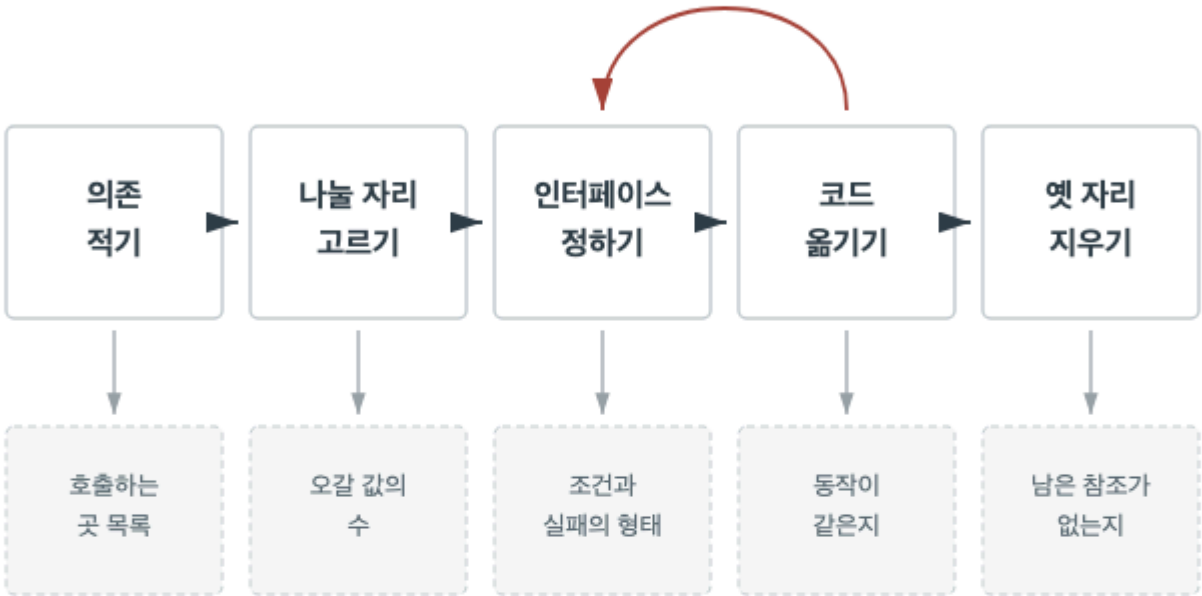
먼저 적어 두니 옮길 범위가 저절로 정해졌다. 이 두 함수를 만드는 데 필요한 코드만 새 모듈로 가고 나머지는 남는다. 무엇을 옮길지 판단이 서지 않던 코드가 몇 개 나왔는데, 두 함수 어느 쪽에도 쓰이지 않는다는 이유로 남겼다.

인터페이스를 나중에 정했다면 옮긴 코드의 모양을 따라 인터페이스가 정해졌을 것이다. 그러면 지금의 구현 방식이 그대로 약속이 되어, 다음에 문구 만드는 방식을 바꿀 때 사용하는 쪽까지 고쳐야 한다. 순서를 뒤집는 것만으로 이 문제를 피했다.

분리 절차와 확인 지점

코드를 옮기는 일은 넷째 자리에서 시작한다

오갈 값이 정리되지 않으면 되돌아간다



각 단계는 확인 항목을 지나야 다음으로 간다

6.6 옮긴 뒤 확인한 것

코드를 옮긴 뒤 확인한 것은 세 가지였다. 첫째, 옮기기 전에 적어 둔 호출 목록의 모든 자리가 새 인터페이스만 쓰고 있는지 보았다. 옛 함수를 직접 부르는 자리가 둘 남아 있어 함께 고쳤다.

둘째, 앞 절에서 찾아 둔 숨은 의존 둘을 다시 보았다. 함께 읽고 쓰던 설정 값은 새 모듈이 인자로 받도록 바꿨고, 순서 약속은 주문 저장에 끝난 결과를 인자로 넘기게 만들어 순서가 코드에 드러나게 했다. 이 두 가지가 남았다면 나눈 뒤에도 두 모듈은 함께 고쳐졌을 것이다.

셋째, 옛 자리에 남은 코드가 없는지 확인했다. 옮긴 코드를 지우지 않고 남겨 두면 어느 쪽이 실제로 쓰이는지 다음 사람이 알 수 없다. 쓰이지 않는 사본은 시간이 지나면 한쪽만 고쳐진 채로 남는다.

6.7 남은 결합과 다음 단계

분리 뒤에도 남은 결합이 있었다. 새 모듈은 문구를 만들 때 주문 상태의 이름을 그대로 썼는데, 이 이름은 주문 모듈에서 정한 것이다. 주문 쪽에서 상태를 하나 늘리면 문구 쪽도 함께 고쳐야 한다. 값을 통해 묶인 결합이므로 다루기 어려운 종류는 아니지만 남아 있는 것은 사실이다.

이 결합을 지금 없애지 않기로 한 것은 상태 목록이 지난 이력에서 거의 바뀌지 않았기 때문이다. 자주 바뀌지 않는 것에 매인 결합은 우선순위가 낮다. 남길 결합과 없앨 결합을 가르는 기준은 결합의 존재가 아니라 그 결합이 실제로 수정을 부르는 빈도다.

다음 단계로는 재고 처리 분리를 후보에 두었다. 다만 오가는 값이 많아 인터페이스 초안부터 다시 만들어야 하고, 그 전에 이번 분리로 주문 모듈의 수정 횟수가 실제로 줄었는지 확인하기로 했다.

7.1 과도한 분해의 비용

나누는 일에도 비용이 있다. 함수가 잘게 나뉘면 한 처리를 따라가는 데 여러 자리를 오가야 하고, 각 함수의 이름만 보고는 전체 순서를 알 수 없어 결국 모두 열어 보게 된다. 읽는 사람이 머릿속에 담아야 할 이름의 수도 함께 는다.

모듈 단위에서는 비용이 더 크다. 모듈이 늘면 그 사이의 인터페이스가 늘고, 인터페이스는 한 번 정해지면 바꾸기 어렵다. 한 가지 요구가 바뀔 때 세 모듈의 인터페이스를 함께 고쳐야 하는 상태는 나누기 전보다 나쁘다.

나누는 일이 이득이 되는 경우는 나뉜 각 부분을 따로 읽고 따로 고칠 수 있을 때다. 나눈 뒤에도 늘 함께 열어야 한다면 그 분해는 자리를 늘렸을 뿐 경계를 만들지 못한 것이다. 분해의 성과는 개수가 아니라 따로 다룰 수 있게 된 단위의 수로 센다.

7.2 분해를 멈추는 기준

분해를 멈출 지점은 세 물음으로 정한다. 이 단위에 이름을 붙일 수 있는가, 이 단위가 바뀌는 이유가 하나인가, 이 단위를 따로 확인할 수 있는가. 셋 다 그렇다면 더 나눌 이유가 없다. 하나라도 아니라면 나눌 후보가 남아 있는 것이다.

반대로 멈춰야 한다는 신호도 있다. 나눈 뒤 이름을 짓기 어려워 순서를 뜻하는 이름을 붙이게 되거나, 나뉜 부분이 언제나 같은 순서로만 불리거나, 나눈 뒤 넘길 값이 오히려 늘었다면 이미 지나친 것이다. 이때는 되돌리는 편이 낫다.

분해는 한 번에 끝내는 작업이 아니다. 지금 알고 있는 만큼만 나누고, 요구가 바뀌어 새 경계가 보이면 그때 다시 나눈다. 미리 나뉜 둔 경계는 대개 실제 변경의 결과 나뉘는 자리와 다르다.

7.3 인터페이스를 넓힐 때와 좁힐 때

인터페이스를 넓히는 것은 새 약속을 더하는 일이다. 더한 약속은 사용하는 쪽이 의지하기 시작하면 되돌리기 어려우므로, 넓힐 때는 그 항목이 여러 사용처에서 필요한지 먼저 확인한다. 한 곳에서만 필요하다면 그 자리에서 해결하고 약속으로 만들지 않는다.

좁히는 것은 이미 준 약속을 거두는 일이라 더 조심스럽다. 쓰는 자리를 모두 찾아 대신할 방법을 마련한 뒤에야 지울 수 있다. 그래서 처음부터 좁게 시작하는 편이 낫고, 확신이 없는 항목은 공개하지 않고 두는 편이 낫다.

판단 기준은 한 가지다. 이 항목이 사용하는 쪽의 일에 필요한가, 아니면 지금 구현이 그렇게 되어 있어서 드러난 것인가. 구현 사정으로 드러난 항목은 구현이 바뀔 때 함께 흔들리므로 약속에 넣지 않는다. 계약이 구현보다 오래 남으려면 계약에 구현이 들어 있지 않아야 한다.

7.4 결합과 응집 점검 항목

구조를 점검할 때는 다음 항목을 차례로 본다. 첫째, 한 가지 요구가 바뀔 때 열어야 하는 모듈이 몇 개인가. 하나에 가까울수록 좋고 셋을 넘으면 경계를 다시 볼 자리다. 둘째, 의존 화살표가 한 방향으로 흐르는가. 돌아오는 화살표가 있으면 그 묶음은 하나의 단위다.

셋째, 각 모듈의 이름이 안에 든 것을 다 설명하는가. 이름에 '기타'나 '도구'가 들어가면 응집이 낮다. 넷째, 모듈 사이에 오가는 값에 상대의 내부 구조가 들어 있는가. 들어 있다면 인터페이스는 있지만 실제로는 구현에 매여 있다.

다섯째, 호출로 드러나지 않는 의존이 있는가. 함께 읽고 쓰는 값과 지켜야 하는 호출 순서가 여기 든다. 이 다섯 항목은 코드를 실행하지 않고도 확인할 수 있으며, 어느 하나에 걸리면 앞의 장에서 다룬 방법 가운데 대응하는 것을 찾아 적용한다.

7.5 경계를 바꾸는 순서

이미 돌아가는 구조를 고칠 때는 한 번에 큰 폭으로 바꾸지 않는다. 큰 변경은 문제가 생겼을 때 어느 부분 때문인지 가릴 수 없게 만든다. 대신 지금 상태를 적고, 한 자리를 떼고, 동작이 같은지 확인하고, 다음 자리로 넘어가는 방식으로 진행한다.

떼는 순서는 자주 바뀌면서 오가는 값이 적은 쪽부터다. 자주 바뀌는 쪽을 먼저 떼면 나머지의 수정 횟수가 곧바로 줄어 다음 작업이 쉬워지고, 오가는 값이 적으면 인터페이스를 정하는 데 드는 시간이 짧다. 두 조건이 어긋나면 값이 적은 쪽을 먼저 한다.

각 단계가 끝날 때마다 남은 결합을 적어 둔다. 없앨 것과 남길 것을 그때그때 정하지 않으면 다음 단계에서 같은 판단을 다시 하게 된다. 남긴 이유까지 적어 두면 나중에 그 이유가 유효한지만 확인하면 된다.

7.6 설계 결정을 남기는 방법

구조에 대한 결정은 코드만 보아서 알 수 없다. 왜 이 자리에 경계를 그었는지, 왜 저 결함을 남겨 두었는지는 결과에 드러나지 않는다. 그래서 결정을 내릴 때 근거를 함께 남긴다. 남기지 않으면 다음 사람은 같은 검토를 처음부터 다시 하거나, 이유 없이 그렇게 된 줄 알고 되돌린다.

남길 것은 길지 않아도 된다. 무엇을 나누었는지, 나눈 기준이 무엇이었는지, 검토했지만 고르지 않은 안이 무엇이고 왜 고르지 않았는지, 지금 남아 있는 결함이 무엇인지면 충분하다. 특히 고르지 않은 안을 적어 두면 다음에 같은 자리를 다시 볼 때 시작점이 달라진다.

기록은 인터페이스 옆에 두는 것이 좋다. 계약과 그 계약을 그렇게 정한 이유가 가까이 있으면 계약을 고칠 때 이유도 함께 갱신된다. 멀리 떨어진 기록은 갱신되지 않고 남아 오히려 잘못된 근거가 된다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 하나다. 무엇을 경계 밖에 두고 무엇을 안쪽에 감출 것인가. 함수에서는 이름과 입력과 결과가 경계였고, 모듈에서는 공개한 함수의 목록과 그 계약이 경계였다. 규모는 달라도 판단 방법은 같았다.

경계를 정하는 기준도 하나로 모인다. 함께 바뀌는 것을 한자리에 두고, 다른 이유로 바뀌는 것을 나누며, 그 사이에 오가는 것을 적게 유지한다. 결합과 응집은 이 기준을 두 방향에서 부른 이름이고, 인터페이스와 계약은 그 경계를 지키는 수단이다.

마지막으로 남길 것은 이 판단이 한 번에 끝나지 않는다는 점이다. 요구가 바뀌면 함께 바뀌는 것의 묶음도 바뀌고, 그러면 어제 옳던 경계가 오늘은 아니다. 좋은 구조는 완성된 배치가 아니라 바뀐 요구에 맞춰 경계를 다시 그을 수 있는 상태를 뜻한다.